

Full-Stack Jewellery Raw Material & Vendor Management Web Application

1. Project Identity

Project Name: JewelFlow — Jewellery Raw Material & Vendor Management System
Type: Full-Stack Web Application (Client Demo Version)
Target Users: Jewellery business owners, admin staff, and production managers
Deployment: Cloud-ready, responsive across desktop, tablet, and mobile

2. Core Objective

Build a **minimal, intuitive, and professional** full-stack web application that enables jewellery businesses to seamlessly manage:

- Gold & Silver raw material purchases
- Vendor-wise raw material distribution and tracking
- Jewellery production monitoring
- Pending material calculations with automated business logic
- Overdue reminders and alerts
- Dashboard reporting with real-time analytics

The application must feel **premium yet simple**, designed specifically for non-technical users in the jewellery industry, with **Hindi-to-English toggle support** for accessibility across Indian markets.

3. Technology Stack (Strictly As Per SOW)

Layer	Technology	Justification
Frontend	React.js 18+ (with Next.js 14 App Router)	SSR for SEO, fast routing, professional UI
Backend	Node.js (Express.js / NestJS)	Scalable REST API, async handling
Database	PostgreSQL 15+	Relational integrity for inventory, ACID compliance
ORM	Prisma ORM	Type-safe database queries, migration management
Authentication	JWT (jsonwebtoken) + bcrypt	Stateless, secure, role-based access
State Management	Zustand / React Query	Lightweight, minimal boilerplate
UI Library	Tailwind CSS + shadcn/ui	Rapid, consistent, accessible components
Charts & Reports	Recharts / Tremor	Lightweight, responsive data visualization
Translation	react-i18next	Robust i18n with Hindi (hi) and English (en)

Table 1 – continued

Layer	Technology	Justification
Icons	LucideReact	Clean,professionaliconography
Fonts	Inter(body)+PlayfairDisplay (headers)	Professional,premiumfeel
Hosting	VPS/CloudServer(AWS/ DigitalOcean)	Production-readydeployment
Security	Helmet.js,CORS,RateLimiting, SSL	Enterprise-gradeprotection
Backup	AutomatedPostgreSQLdumps	Dailybackupsystem

4. Design System & Aesthetic Guidelines

4.1 Color Palette (Eye-Soothing Light Theme)

Token	Hex	Usage
Primary	#2C5F7C	Headers, primary buttons, active states _deep teal-blue, trust & professionalism
Primary Light	#E8F4F8	Hover backgrounds, subtle highlights
Secondary	#D4A57	Gold accents, jewellery-themed highlights, premium feel
Secondary Light	#FAF3E8	Card backgrounds, warm sections
Background	#F8FAFC	Main app background _cool, clean, easy on eyes
Surface	FFFFFF	Cards, modals, input fields
Surface Elevated	#F1F5F9	Tables, secondary containers
Text Primary	#1E293B	Headings, primary text _high contrast but not harsh black
Text Secondary	#64748B	Labels, descriptions, placeholders
Text Muted	#94A3B8	Disabled states, timestamps
Success	#10B981	Positive stock, completed transactions
Warning	#F59E0B	Due soon, pending alerts
Danger	#EF4444	Overdue, negative balance, errors
Border	#E2E8F0	Subtle dividers, input borders
Gold Metallic	#C9A96E	Special jewellery-related badges, purity indicators
Silver Metallic	#A8B5C4	Silver material indicators

4.2 Typography (Professional & Readable)

- **Primary Font:** Inter (Google Fonts) — clean, modern, excellent for data-heavy interfaces
- **Display Font:** Playfair Display (Google Fonts) — elegant, used sparingly for dashboard headings and brand elements

- **Hindi Font:** Noto Sans Devanagari — seamless Hindi script rendering
- **Scale:**
 - H1: 28px / 700 / tracking-tight
 - H2: 22px / 600
 - H3: 18px / 600
 - Body: 14px / 400 / line-height 1.6
 - Small: 12px / 500
 - Data/Numbers: 16px / 600 / tabular-nums (for aligned weight calculations)

4.3 UI Principles

- **Minimalism:** Every screen should have ONE primary action. No clutter.
- **Whitespace:** Generous padding (24px base), breathable layouts
- **Rounded Corners:** `rounded-lg` (8px) for cards, `rounded-md` (6px) for buttons
- **Shadows:** Subtle `shadow-sm` for cards, `shadow-md` for modals — never heavy shadows
- **Transitions:** Smooth 200ms transitions on all interactive elements
- **Data Density:** Tables should use compact rows (48px height) with zebra striping (`Surface Elevated alternate`)
- **Mobile-First:** Sidebar collapses to bottom nav on mobile, tables become horizontal-scroll cards

5. Functional Modules (Detailed Specifications)

5.1 Authentication & User Management

Requirements:

- Secure Login page with email/username + password
- Role-based access control: **Admin** (full access) and **User** (view-only / limited edit)
- JWT token stored in httpOnly cookie (secure, XSS-proof)
- Auto-logout after 30 minutes of inactivity
- Password reset via email token
- Login page should have bilingual toggle (Hindi/English) visible

UI Details:

- Centered card layout on light background
- Logo + “JewelFlow” branding at top
- Clean input fields with icons
- “Remember Me” checkbox
- “Forgot Password?” link

5.2 Dashboard (Home Screen)

Requirements:

- Real-time summary cards showing:
 - **Total Stock** (Gold + Silver separate cards) — calculation: $\text{Purchased} - \text{Issued} + \text{Returned}$
 - **Total Pending with Vendors** — sum of all vendor pending balances
 - **Total Jewellery Produced** — sum of all net weights received
 - **Overdue Reminders Count** — items past expected return date
- Interactive charts:
 - Line chart: Monthly material purchase vs issue trend (last 6 months)
 - Pie chart: Vendor-wise pending material distribution
 - Bar chart: Monthly jewellery production (net weight)
 - Recent Activity Feed: Last 10 transactions (purchase/issue/receive)
 - Overdue Alerts Banner: Prominent red banner if any overdue items exist

UI Details:

- Grid layout: 4 summary cards on top (responsive: 2x2 on mobile)
- Cards have subtle gradient backgrounds using Primary Light / Secondary Light
- Charts use brand colors only (Primary, Secondary, Success, Warning, Danger)
- “Refresh Data” button with last-updated timestamp

5.3 Raw Material Management

Requirements:

- **Purchase Entry Form:**
 - Material Type: Gold / Silver (dropdown)
 - Purity: 24K, 22K, 18K, 14K for Gold; 925, 999 for Silver (dropdown)
 - Gross Weight (grams) — numeric input with 3 decimal precision
 - Net Weight auto-calculated if purity selected (optional logic)
 - Vendor/Supplier Name
 - Purchase Date (default today)
 - Rate per gram (INR)
 - Total Amount (auto-calculated: $\text{Weight} \times \text{Rate}$)
 - Invoice Number (optional)
 - Notes
- **Inventory List:**
 - Table view with columns: Date, Material, Purity, Weight, Rate, Amount, Actions
 - Search by vendor name or invoice

- Filter by material type and date range
- Sort by date (newest first)
- Edit/Delete actions (Admin only)
- **Stock Summary:**
 - Running total: Current Stock = Previous Stock + Purchased Stock
 - Separate tracking for each purity grade
 - Low stock warning if below configurable threshold

Calculation Logic:

Current Stock = Previous Stock + Purchased Stock Pure Gold = Net Weight × (Purity / 24) [for 24K standard]

UI Details:

- Form in a modal or dedicated page with card layout
- Weight inputs show “grams” unit suffix
- Auto-calculate Total Amount on blur
- Inventory table has color-coded material badges (Gold = Gold Metallic, Silver = Silver Metallic)

5.4 Vendor Management

Requirements:

- **Vendor Profile Creation:**
 - Vendor Name (Hindi + English support)
 - Contact Person
 - Phone Number (Indian format validation)
 - Address
 - Specialization (e.g., “Casting”, “Setting”, “Polishing”)
 - Status: Active / Inactive
- **Vendor List:**
 - Card grid view (preferred over table for profiles)
 - Each card shows: Name, Phone, Specialization, Active Status badge
 - Click to view full profile + transaction history
- **Vendor Transaction History:**
 - Tabbed view: “Material Issued” | “Jewellery Received” | “Raw Material Returned”
 - Chronological list with weights and dates
 - Running balance per vendor

UI Details:

- Vendor cards have avatar placeholder with first letter
 - Active = green dot, Inactive = gray dot
 - Transaction history uses timeline-style layout on desktop
-

5.5 Raw Material Distribution (Issue to Vendor)

Requirements:

- **Material Issue Form:**
 - Select Vendor (searchable dropdown)
 - Select Material from Current Stock (shows available weight per purity)
 - Issue Weight (validated against available stock — cannot exceed)
 - Purity (auto-filled from selected stock)
 - Expected Return Date (date picker — default +15 days)
 - Purpose: “Making Order” / “Sample” / “Repair”
 - Notes
- **Available Stock Display:**
 - Before issue, show real-time available stock
 - After issue: $\text{Available Stock} = \text{Current Stock} - \text{Issued Material}$
- **Issue Register:**
 - Table: Date, Vendor, Material, Purity, Issued Weight, Expected Return, Status (Pending/Returned/Overdue)
 - Color-coded status badges

Calculation Logic:

$$\text{Available Stock} = \text{Current Stock} - \text{Issued Material}$$
$$\text{Pending Material} = \text{Available Issued} - \text{Material Received Back}$$

UI Details:

- Stock validation: red warning if issue weight > available
 - Expected return date shows calendar with highlighted date
 - Status badges: Pending (blue), Returned (green), Overdue (red with pulse animation)
-

5.6 Making Jewellery Panel (Receive from Vendor)

Requirements:

- **Jewellery Receive Form:**

- Select Vendor (shows only vendors with pending material)
- Select Original Issue Entry (link to track consumption)
- Gross Weight of received jewellery (grams)
- Stone Weight (grams) — if applicable
- Net Weight auto-calculated: $\text{Net} = \text{Gross} - \text{Stone}$
- Wastage auto-calculated: $\text{Wastage} = \text{Issued Material} - \text{Net Weight}$
- Wastage % auto-calculated: $(\text{Wastage} / \text{Issued Material}) \times 100$
- Wastage tolerance check: if > 10%, highlight in red for review
- Returned Raw Material weight (if vendor returns unused material)
- Product Description / Item Name
- Received Date
- Quality Remarks
- Receive Register:
- Table view with all calculations visible
- Filter by vendor and date range

Calculation Logic:

```

Net Weight = Gross Weight - Stone Weight
Wastage = Issued Material - Net Weight
Wastage % = (Wastage / Issued Material) × 100
Vendor Balance = Issued Material - (Received Jewellery Net Weight + Returned Raw Material)

```

UI Details:

- Auto-calculation happens in real-time as user types
- Wastage percentage shown in a circular progress indicator
- Red alert if wastage exceeds 10%
- Net weight displayed prominently in bold

5.7 Reminder & Notification System

Requirements:

- Overdue Detection Logic:
- Daily background job (or real-time check on login)
- If $\text{Current Date} > \text{Expected Return Date}$ AND $\text{Pending Balance} > 0 \rightarrow \text{Status} = \text{OVERDUE}$
- Notification Types:
- Due Soon: 2 days before expected return (yellow badge)
- Overdue: Past expected return (red badge + email/SMS placeholder)

- Low Stock: When any material stock below threshold
- Notification Center:
 - Bell icon in header with badge count
 - Dropdown showing recent notifications
 - Mark as read / dismiss functionality
- Dashboard Alerts:
 - Persistent banner for overdue items
 - Count badge on “Reminders” sidebar item

UI Details:

- Bell icon with red dot when unread notifications exist
- Notification items have timestamp and direct link to relevant transaction
- Overdue items in tables have red row background or left border accent

5.8 Reports & Analytics

Requirements:

- Stock Summary Report:
 - Opening Balance + Purchases - Issues + Returns = Closing Balance
 - Filter by material, purity, date range
 - Export to PDF / Excel (placeholder for demo)
- Vendor-wise Pending Report:
 - List all vendors with:
 - Total Issued
 - Total Received (Net)
 - Total Returned
 - Pending Balance
 - Wastage Amount & %
 - Sort by pending balance (highest first)
- Production Report:
 - Monthly/Quarterly jewellery production
 - Item-wise breakdown
 - Wastage trend analysis
- Material Ledger:
 - Complete transaction history per material type

- Debit (purchase/return) and Credit (issue) format

UI Details:

- Reports page has filter sidebar + main content area
- Print-friendly styling (white background, clean borders)
- “Export” button with dropdown: PDF, Excel, Print
- Charts embedded in reports for visual summary

6. Hindi English Translation Feature (i18n)

Implementation:

- Use `react-i18next` with `i18next` backend
- Language files: `en.json` and `hi.json`
- Language toggle: Toggle switch in header (EN / हिंदी)
- All UI text, labels, placeholders, buttons, error messages must be translated
- Number & Date Formatting:
 - Hindi: DD/MM/YYYY date format, Indian number system (₹1,00,000)
 - English: DD/MM/YYYY or configurable
- Key Areas for Hindi:
 - Sidebar navigation labels
 - Form field labels and placeholders
 - Button text (Save, Cancel, Delete, Edit)
 - Table headers
 - Dashboard card titles
 - Notification messages
 - Error messages and validation text
 - Report titles and column headers

Sample Translation Keys:

```
// en.json
{
  "nav.dashboard": "Dashboard",
  "nav.materials": "Raw Materials",
  "nav.vendors": "Vendors",
  "nav.issue": "Issue Material",
  "nav.receive": "Receive Jewellery",
  "nav.reports": "Reports",
  "common.save": "Save",
  "common.cancel": "Cancel",
```

```

"common.delete": "Delete",
"common.edit": "Edit",
"common.search": "Search...",
"material.gold": "Gold", "material.silver":
"Silver", "weight.grams": "grams",
"status.pending": "Pending",
"status.overdue": "Overdue",
"status.completed": "Completed"
}

// hi.json
{
  "nav.dashboard": "",
  "nav.materials": "",
  "nav.vendors": "",
  "nav.issue": "",
  "nav.receive": "",
  "nav.reports": "",
  "common.save": "",
  "common.cancel": "",
  "common.delete": "",
  "common.edit": "",
  "common.search": "...",
  "material.gold": "",
  "material.silver": "",
  "weight.grams": "",
  "status.pending": "",
  "status.overdue": "",
  "status.completed": ""
}

```

UI Behavior:

- Language preference saved in localStorage
- Instant switch without page reload
- Hindi text should not break layout (test with longest translations)
- Right-to-left considerations not needed (Hindi is LTR script)

7. Database Schema (PostgreSQL with Prisma)

Core Tables:

```

model User {
  id String password @id @default(autoincrement())
  email String Role @unique

  name
  role @default(USER)
}

```

```

    createdAt DateTime @default(now())
}

model Vendor {
  id          Int    String @id @default(autoincrement())
  name        String?
  nameHi      String?    // Hindi name
  contact     String?
  phone       String?
  address     String?
  specialty   Boolean
  isActive    DateTime   @default(true)
  createdAt   @default(now())
  issues      MaterialIssue[]
  receives    JewelleryReceive[]
}

model RawMaterialPurchase {
  id          Int    @id @default(autoincrement())
  material     MaterialType
  purity       String // "24K", "22K", "925", etc.
  grossWeight  Decimal @db.Decimal(10, 3)
  netWeight    Decimal @db.Decimal(10, 3)
  ratePerGram  Decimal @db.Decimal(10, 3)
  totalAmount  Decimal @db.Decimal(10, 2)
  @db.Decimal(12, 2)
  vendorName   String?
  invoiceNo    String?
  purchaseDate DateTime @default(now())
  notes        String?
}

model MaterialIssue { Int Int
  id          Int    @id @default(autoincrement())
  vendorId    Int
  vendor      @relation(fields: [vendorId], references:[id])
  materialType
  String
  String
  grossWeight  Decimal @db.Decimal(10, 3)
  expectedReturn DateTime
  purpose      String?
  issueDate    DateTime@default(now())
  status       IssueStatus@default(PENDING)
  notes        String?
  receives     JewelleryReceive[]
}

model JewelleryReceive {
  id          Int    @id @default(autoincrement())
  vendorId    Int Vendor@relation(fields: Int
  vendor      [vendorId], references:[id])
  issueId

```

```

    issue      itemName  MaterialIssue@relation(fields: String    [issueId],references:[id])
    grossWeight      Decimal@db.Decimal(10,
    stoneWeight      Decimal@db.Decimal(10,    3) 3)@default(0) 3)
    netWeight wastage  Decimal@db.Decimal(10,    3)
    wastagePercent    Decimal@db.Decimal(10,
                                Decimal    @db.Decimal(5,    2)
    returnedMaterial  Decimal @db.Decimal(10, 3) @default(0)
    receiveDate       DateTime@default(now())
    qualityRemarks    String?
}

enum Role {
    ADMIN
    USER
}

enum MaterialType {
    GOLD
    SILVER
}

enum IssueStatus {
    PENDING
    RETURNED
    OVERDUE
}

```

Views/Queries:

- **Stock View:** Aggregated by material + purity
- **Vendor Balance View:** SUM(issued) - SUM(received + returned)
- **Overdue View:** issueDate > expectedReturn AND status = PENDING

8. API Endpoints (RESTful)

Authentication

- POST /api/auth/login — JWT login
- POST /api/auth/logout — Clear cookie
- GET /api/auth/me — Current user
- POST /api/auth/reset-password — Request reset

Dashboard

- GET /api/dashboard/summary — Stock, pending, production, overdue counts
- GET /api/dashboard/trends GET — Chart data (monthly)
- /api/dashboard/recent — Recent 10 transactions

Raw Materials

- GET /api/materials — List all purchases (with filters)
- POST /api/materials — Create purchase
- PUT /api/materials/:id — Edit purchase
- DELETE /api/materials/:id — Soft delete
- GET /api/materials/stock — Current stock summary

Vendors

- GET /api/vendors — List all vendors
- POST /api/vendors — Create vendor
- PUT /api/vendors/:id — Update vendor
- GET /api/vendors/:id — Vendor detail + history
- GET /api/vendors/:id/balance — Running balance

Material Issue

- GET /api/issues — List issues (with status filter)
- POST /api/issues — Create issue (validate stock!)
- PUT /api/issues/:id — Update expected date / status
- GET /api/issues/overdue — All overdue items

Jewellery Receive

- GET /api/receives — List received items
- POST /api/receives — Create receive entry (auto-calculate wastage)
- GET /api/receives/:id — Detail view

Rep orts

- GET /api/reports/stock — Stock ledger
- GET /api/reports/vendor-pending — Vendor-wise pending
- GET /api/reports/production — Production summary
- GET /api/reports/wastage — Wastage analysis

Notifications

- GET /api/notifications — User notifications
- PUT /api/notifications/:id/read — Mark read
- GET /api/notifications/unread-count — Badge count

9. Business Logic & Calculation Engine

Automated Calculations (Server-Side Validation Required):

1. Purchase Entry:

$$\text{totalAmount} = \text{grossWeight} \times \text{ratePerGram}$$
$$\text{netWeight} = \text{grossWeight} \text{ (if purity = 24K), else } \text{grossWeight} \times (\text{purity}/24)$$

2. Stock Calculation:

```
currentStock[material][purity] = SUM(purchases) - SUM(issues) + SUM(returns)
```

3. Material Issue Validation:

```
IF issuedWeight > currentStock[material][purity]:  
  RETURN error "Insufficient stock"
```

4. Jewellery Receive:

```
netWeight = grossWeight - stoneWeight  
wastage = originalIssuedWeight - netWeight - returnedMaterial  
wastagePercent = (wastage / originalIssuedWeight) × 100  
vendorBalance = issuedMaterial - (SUM(allReceives.netWeight) + SUM(allReturns))
```

5. Overdue Detection:

```
IF today > expectedReturnDate AND issue.status == "PENDING":  
  issue.status = "OVERDUE"  
  CREATE notification
```

6. Dashboard Aggregations:

```
totalStock = SUM(allPurchases.netWeight) - SUM(allIssues.issuedWeight) + SUM(allReturns)  
totalPending = SUM(vendorBalances where balance > 0)  
totalProduced = SUM(allReceives.netWeight)  
overdueCount = COUNT(issues where status = "OVERDUE")
```

10. Page Structure & Navigation

Sidebar Navigation (Collapsible):

- 1. **Dashboard** — LayoutDashboard icon
- 2. **Raw Materials** — Package icon
 - Purchase Entry
 - Inventory List
 - Stock Summary
- 3. **Vendors** — Users icon
 - Vendor List
 - Add Vendor
- 4. **Issue Material** — ArrowUpRight icon
 - New Issue
 - Issue Register
- 5. **Receive Jewellery** — ArrowDownLeft icon
 - New Receive
 - Receive Register
- 6. **Reports** — BarChart3 icon
 - Stock Report
 - Vendor Pending

- Production Report
- Material Ledger

7. **Reminders** — Bell icon (with count badge)

8. **Settings** — Settings icon

- Language Toggle
- Profile
- Logout

Header (Top Bar):

- Hamburger menu (mobile)
- App logo + “JewelFlow”
- LanguageToggle:[EN] 1
- Notification Bell with badge
- User avatar dropdown (Profile / Logout)

11. Demo Data for Client Presentation

Pre-populate with realistic sample data:

Purchases (5 entries):

- Gold 24K — 100g — ₹5,800/g — Total ₹5,80,000
- Gold 22K — 250g — ₹5,400/g — Total ₹13,50,000
- Silver 925 — 500g — ₹72/g — Total ₹36,000

Vendors (4 entries):

- Raju Jewellers (Casting)
- Sharma & Sons (Stone Setting)
- GoldCraft Works (Polishing)
- Mehta Art Jewellers (Designing)

Issues (6 entries):

- Issue 50g 22K Gold to Raju — Expected return: 10 days ago (make overdue for demo)
- Issue 30g 24K Gold to Sharma — Expected return: tomorrow (due soon demo)
- Issue 100g Silver to GoldCraft — Expected return: next week

Receives (3 entries):

- Received from Raju: Gross 48g, Stone 2g, Net 46g, Wastage 4g (8%)
- Received from Sharma: Gross 29g, Stone 0.5g, Net 28.5g, Wastage 1.5g (5%)

12. Responsive Breakpoints

Breakpoint	Width	Layout Changes
Mobile	< 640px	Bottom nav, stacked cards, horizontal scroll tables
Tablet	640-1024px	Collapsible sidebar, 2-column grids
Desktop	> 1024px	Full sidebar, 3-4 column grids, side-by-side forms

13. Security & Performance Requirements

- **Input Validation:** All forms validated server-side (Zod schema)
- **SQL Injection Protection:** Prisma ORM parameterized queries
- **XSS Protection:** Helmet.js headers, sanitized inputs
- **CSRF Protection:** Cookie-based JWT with SameSite=strict
- **Rate Limiting:** 100 requests/minute per IP, 10 login attempts per 5 minutes
- **Password Policy:** Min 8 chars, 1 uppercase, 1 number, 1 special char
- **Data Encryption:** Passwords hashed with bcrypt (salt rounds 12)
- **Backup:** Daily automated PostgreSQL dump to secure storage
- **Performance:** API response < 200ms, page load < 2s, Lighthouse score > 90

14. Deployment Checklist

- Environment variables configured (DB_URL, JWT_SECRET, etc.)
- PostgreSQL database initialized with Prisma migrate
- Demo data seeded via seed script
- SSL certificate installed
- PM2/systemd configured for Node.js process
- Nginx reverse proxy setup
- Static assets served via CDN (optional)
- Error logging (Sentry/LogRocket) integrated
- Health check endpoint (GET /api/health) active

15. Deliverables for Client Demo

1. **Live Application URL** — Fully functional demo deployed on VPS
2. **Admin Credentials** — Pre-configured admin login
3. **User Credentials** — Pre-configured read-only user login
4. **Demo Data** — Realistic jewellery business data pre-loaded
5. **Feature Walkthrough:**
 - Login → Dashboard overview
 - Add Purchase → View Stock Update
 - Issue Material → Vendor Tracking
 - Receive Jewellery → Auto Wastage Calculation
 - View Overdue Reminders

- Generate Reports
- Switch Language (Hindi English)

6. **Source Code** — Clean, commented, well-structured repository

7. **README.md** — Setup instructions, environment variables, seed commands

16. Success Criteria

- All 8 functional modules working end-to-end
 - Calculations accurate to 3 decimal places for weights
 - Hindi translation covers 100% of UI text
 - Overdue reminder triggers automatically
 - Dashboard loads with real aggregated data in < 2 seconds
 - Application works smoothly on mobile browsers
 - No console errors, no broken layouts in either language
 - Professional appearance suitable for client presentation
 - Demo data tells a convincing “day in the life” story
-

End of Prompt Document — JewelFlow Development Specification